



**Trabajo Práctico 3**  
**Programación III - COM-02**  
**Primer cuatrimestre 2026**

**Profesores:**

- Gabriel Carrillo
- Gonzalo Faccini

**Alumnos:**

- Emiliano Acosta:
- Santiago Ruiz:
- Matias Rios Alcaray:

**Fecha de entrega:**

Lunes 11 de junio de 2026.

# Implementación

El trabajo práctico se buscó utilizar todos los conceptos de la programación orientada a objetos (POO) que vimos durante la carrera y tener en cuenta el concepto de “separated presentation”, es decir, el código que se utiliza para que la interfaz esté correctamente encapsulada y separada del código de negocio que ejecuta las funciones esenciales del programa, utilizando una arquitectura de diseño de 3 capas (model-view-presenter).

A partir de nuestra elección de arquitectura, se optó por realizar una división del código en tres paquetes: por un lado tenemos un paquete de GUI con la interfaz de usuario que contiene la clases pertinentes a las distintas ventanas del juego, otro paquete de la lógica con las clases relativas a la lógica de negocio en donde se encuentra una clase que gestiona los algoritmos para el funcionamiento del core de la aplicación, y por último un paquete de Datos en donde se ubican las clases que interviene en el guardado y recuperación de datos de los empleados y equipos generados.

Además en este trabajo entraron los conceptos e implementación de los algoritmos de Fuerza Bruta y Backtracking, la implementación de algunos tipos de heurísticas (goloso) y la utilización de threads para garantizar el poder seguir utilizando el programa mientras este esté resuelve algunos de los algoritmos.

## View

**PantallaInicio:** el inicio en donde el usuario ve los roles de empleados para armar el equipo y puede indicar cuantos de cada uno necesitará en su conformación. Todos los campos deben ser completados o un aviso visual se mostrará como alerta.

**PantallaEquipo:** la ventana principal de resolución del programa, incluye una lista de empleados precargada y una de conflictos, además de poseer dos secciones para agregar nuevos empleados (que se identifican con su DNI) y nuevos conflictos que no se hayan cargado, además de una serie de botones para buscar una solución con diversos métodos de forma individual y una para resolverlo con todas para ejecutar una comparativa.

**PantallaComparativa:** ventana emergente que se abre si se ejecuta la opción para encontrar el equipo con todos los métodos a la vez, muestra tres tablas con la configuración de empleados resultante de cada algoritmo, otra con datos comparativos de cada algoritmo ejecutado y un gráfico jFreeChart para destacar visualmente la exactitud y diferencia entre los ratings promedios resultantes de la solución de cada algoritmo. Además, la clase del Observer correspondiente a esta que es notificado cuando un nuevo equipo o comparativa son generados.

Además de asociar estas pantallas visuales a un presenter para separar la lógica de negocio de lo visual y las pantallas sean tontas.

# Lógica de Negocio

Aquí tenemos una clase principal que se ocupa de la entrada y gestión de los elementos de esta capa llamada GestorEquipo, la cual creamos bajo el patrón de diseño Singleton, en la cual también incluimos el patrón Strategy para la inyección del comparador al algoritmo de heurística y un sistema de notificación a Observers gracias a los cuales se informa la generación de un nuevo equipo a la visual y a la capa de datos. Además una clase abstracta Algoritmo de la cual heredan FuerzaBruta, Backtracking y Heurística los cuales utilizan métodos de recursión para ejecutar su proceso de búsqueda.

En el paquete DAL ubicamos lo pertinente al almacenamiento y recuperación de los empleados con sus datos y los conflictos entre ellos y los equipos generados. Las listas de empleados y sus conflictos se encuentran en dos archivos .json, y hay dos clases de carga y guardado de datos que implementan interfaces para desacoplar y hacer nuestro código reutilizable. La clase saveData es notificada mediante un Observer y guarda en un worklog los equipos generados junto a sus datos y si se ejecutó una comparativa, crea un nuevo archivo .txt con los equipos generados por cada algoritmo y los datos de cada uno.